Git in Practice

From Mental Model to Real Workflow

Session 3 — You already have the map. Today you drive.

M. Souhil | Dev Web Core Team

Data Science Club — USTHB

Tech Department | Dev Web Section

March 9, 2026 | Hands-on Practice Session

Session Roadmap

1. **Quick Recap** — Session 1 in 60 seconds
2. **Git Setup** — config, init, your first repo
3. **Local Workflow** — add, commit, log, diff
4. **GitHub Connection** — remote, push, clone
5. **Branching Workflow** — branch, switch, merge
6. **Pull Requests** — collaborate without chaos
7. **Fixing Mistakes** — undo, restore, amend
8. **Mini Challenge** — real repo from scratch

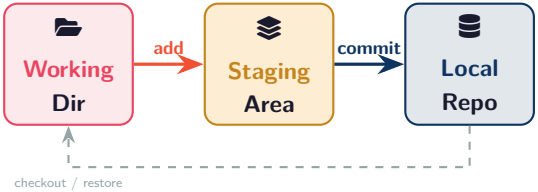
Note

Today is **terminal-first**. Every concept from Session 1 becomes a command you type. Keep a terminal open the whole session.

Part 0 — Quick Recap

Session 1 in 60 Seconds

- **Three areas:** Working Dir → Stage → Repo
- **Commit** = snapshot + hash + parent pointer
- **Branch** = a named pointer, **not** a copy
- **HEAD** = "where you are right now"
- **Git ≠ GitHub** — local engine vs cloud platform
- **DAG** — history never loops backward



Note

If any of these feel fuzzy — re-read Session 1 notes tonight. Everything today **builds on this diagram**.

Part 1 — Git Setup

First Things First — Git Configuration

Key Concept

Before any commits, Git needs to know **who you are**. This info is embedded in every commit you make — forever.

Run these once on your machine:

```
# Set your identity
$ git config -global user.name "Your Name"
$ git config -global user.email "your@mail.com"

# Set default branch name to main
$ git config -global init.defaultBranch main

# Verify everything is saved
$ git config -list
```

Note

`--global` = applies to every repo on your machine. Omit it to set config per-repo only.

Analogy

This is like signing your name on every page before you start writing. Git uses it to label who made each commit — useful when collaborating.

Important

Use the **same email** as your GitHub account — otherwise GitHub cannot link your commits to your profile.

Creating Your First Repository

Option A — Start from scratch (local):

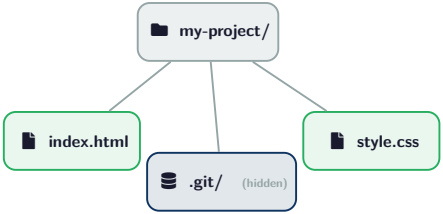
```
$ mkdir my-project
$ cd my-project
$ git init
$ ls -a
# you'll see the .git/ folder
```

Option B — Clone existing repo (remote):

```
$ git clone <your-repo-url>
$ cd repo
```

Note

`git init` creates a hidden `.git/` folder. That folder **is** the repository — delete it and all history is gone.



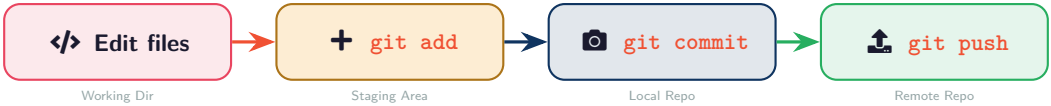
git init creates .git/ — everything else is your files

Analogy

`git init` is like putting a **security camera** in your folder. From this moment on, every change can be recorded.

Part 2 — Local Workflow

The Daily Loop — Add, Commit, Repeat




```
# Stage one file
$ git add index.html

# Stage all changed files
$ git add .

# Commit with a message
$ git commit -m "Add homepage layout"
```

Note

Write good commit messages. A great message completes the sentence: *"If applied, this commit will . . . "*

Good: `"Fix navbar overflow on mobile"`

Bad: `"fix stuff"` `"asdf"`

Seeing What's Happening — Status, Log, Diff

The three commands you'll use most:

```
# What state are my files in?
$ git status
# Show commit history $ git log
$ git log -oneline # compact view
$ git log -oneline -graph # DAG view
# What exactly changed in my files?
$ git diff # unstaged changes
$ git diff -staged # staged vs last
commit
```

Note

`git status` is your **compass**. Run it constantly — before and after every action.

Sample git log –oneline

```
a3f9b2c Add navbar component
7e1b4d8 Fix footer spacing
1f3a9e2 Initial commit
```

Sample git status

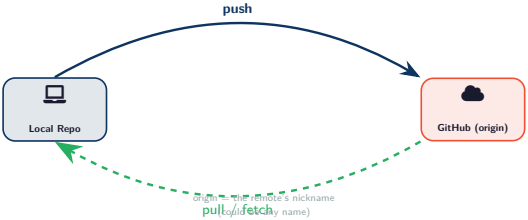
```
On branch main
Changes to be committed:
modified:  style.css
Changes not staged:
modified:  index.html
```

Part 3 — GitHub Connection

Connecting to GitHub — Remote

Key Concept

A **remote** is just a URL that points to a repository hosted elsewhere (e.g. GitHub). The standard name for your main remote is **origin**.



Link your local repo to GitHub:

```
# Add a remote called "origin"
$ git remote add origin
https://github.com/profileName/repo.git

# Verify it's set
$ git remote -v

# Push your commits to GitHub
$ git push -u origin main
```

Analogy

`origin` is just a nickname — like saving a contact as "Home" instead of typing the full address every time.

Note

`-u` sets **upstream**: after this, just type `git push` — no need to repeat `origin main`.

Fetching Changes — Pull vs Fetch

```
# Download + merge remote changes
$ git pull
# Download only (don't merge yet)
$ git fetch
# After fetch:  manually merge
$ git merge origin/main
```

Command	What it does
`git pull`	fetch + merge in one step
`git fetch`	download only; your branch unchanged
`git merge`	integrate fetched changes when ready

Analogy

Fetch = checking your mailbox.
Pull = checking *and* opening every letter immediately.

Sometimes you want to look before you commit to reading everything.

Important

`git pull` on a diverged branch can create an unexpected **merge commit**. Use `git fetch` first if you want to inspect changes before merging.

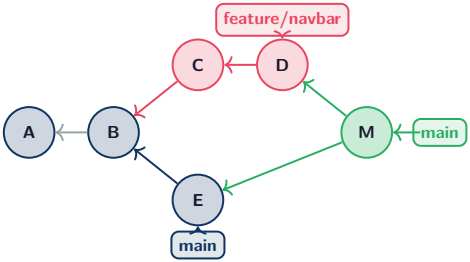
Part 4 — Branching Workflow

Working With Branches

Key Concept

Rule of thumb: Never work directly on `main`. Create a branch for every new feature or fix. Merge when done and reviewed.

```
# See all branches git branch
# Create + switch to new branch
$ git switch -c feature/navbar
# Switch to existing branch
$ git switch main
# Merge feature into main
$ git switch main
$ git merge feature/navbar
# Delete merged branch (cleanup)
git branch -d feature/navbar
```



Note

`git switch -c` = create & switch in one command. Old syntax: `git checkout -b` — still works, same result.

Branch Naming Conventions

Key Concept

Good branch names make the team's history **self-documenting**. You see the purpose without reading every commit.

Name	Use for
`feature/login-page`	New features
`fix/navbar-overflow`	Bug fixes
`hotfix/broken-build`	Urgent production fixes
`chore/update-deps`	Maintenance tasks
`docs/readme-update`	Documentation only

Branch names to avoid:

`my-branch` `test` `temp`
`NEW-FEATURE` `fix123`
`branch for navbar`
`johns_stuff`

Analogy

A branch name is a **ticket title**. Anyone reading `git branch` should immediately know what each branch is for — without asking you.

Important

Avoid spaces, capital letters, and special characters.
Use **lowercase-kebab-case** — always.

Part 5 — Pull Requests

Pull Requests — Collaborate Without Chaos

1. Create branch + commit → 2. Push branch to GitHub → 3. Open Pull Request → 4. Review & discuss → 5. Merge into main

```
# Push your feature branch
$ git push -u origin feature/navbar
# Then open GitHub in your browser:
# Repo → "Compare & pull request"
button
```

A good PR should have:

- ✔ **A clear title** — what does it do?
- ✔ **A description** — why this change?
- ✔ **Small scope** — one thing at a time
- ✔ **No conflicts** — rebase or merge main first
- ✘ **Not:** 47 files and "misc fixes"

Note

A PR is not a Git concept — it lives on **GitHub**. Git only knows about branches and commits. GitHub adds the review layer on top.

Part 6 — Fixing Mistakes

Fixing Mistakes — The Most Useful Commands

Scenario A — Unstage a file (before commit):

```
$ git restore -staged index.html
```

Scenario B — Discard edits in a file:

```
$ git restore index.html
# WARNING: changes are gone permanently
```

Scenario C — Fix the last commit message:

```
$ git commit -amend -m "Better message"
# Only safe BEFORE pushing to remote
```

Scenario D — Undo a commit, keep the changes:

```
$ git reset -soft HEAD 1
# Commit is gone; files stay staged
```

Scenario E — Revert a pushed commit safely:

```
$ git revert <commit-hash>
# Creates a NEW commit that undoes it
# Safe for shared branches
```

Important

Never use `git reset -hard` or `git push -force` on a shared branch.

It rewrites history — your teammates' repos will **break**.

Note

Golden rule: if a commit is already **pushed**, use `revert`. If it's only local, `reset` is safe.

When It All Goes Wrong — ohshitgit

Key Concept

Git's secret weapon: the reflog.
It records **every** action you've taken — even things you thought you deleted.

```
# See the full action history
$ git reflog
# Recover from a bad reset
$ git reset -hard HEAD@2
# Recover a deleted branch
$ git checkout -b recovered HEAD@4
```

Note

The reflog is **local only** — it lives in `.git/logs/`. It does not sync to GitHub.

Sample reflog output

```
a3f9b2c HEAD@{0}:  commit:  Add navbar
7e1b4d8 HEAD@{1}:  reset:  moving to HEAD~1
1f3a9e2 HEAD@{2}:  commit:  Fix footer
9cb3a12 HEAD@{3}:  commit:  Initial commit
```

Analogy

The reflog is Git's **black box recorder**. Even if you crash, the flight data is still there.

`ohshitgit.com` — practical fixes for every "I broke everything" scenario.

Part 7 — Mini Challenge

Mini Challenge — Build a Real Repo 🏆

Note

Solo or in pairs. **15 minutes.** Work entirely in the terminal + GitHub.

1. Create a folder `dsc-challenge/` and run `git init`
2. Create `index.html` with basic HTML boilerplate. Stage and commit: *"Initial commit"*
3. Create a branch `feature/about-page`
4. Add an `about.html` file on that branch. Commit with a meaningful message
5. Switch back to `main`. Merge your feature branch
6. Create a GitHub repo and push `main`
7. **Bonus:** open the repo URL and verify your commits appear correctly

Success looks like this

```
$ git log -oneline
e7c1a3f Merge branch 'feature/about-page'
b2f9d1c Add about page
4a8e2b1 Initial commit
```

Mental checkpoint at each step:

"Which area am I in right now?
What will this command do to my history?"

Note

Raise your hand if you get stuck — come help each other too. Teaching someone else cements your own understanding.

Key Takeaways

Commands you now own:

- `git config` — identity setup
- `git init` / `clone` — start a repo
- `git add` / `commit` — save a snapshot
- `git status` / `log` / `diff` — inspect
- `git push` / `pull` — sync with remote
- `git branch` / `switch` / `merge` — branches
- `git restore` / `revert` — undo safely
- `git reflog` — recover anything

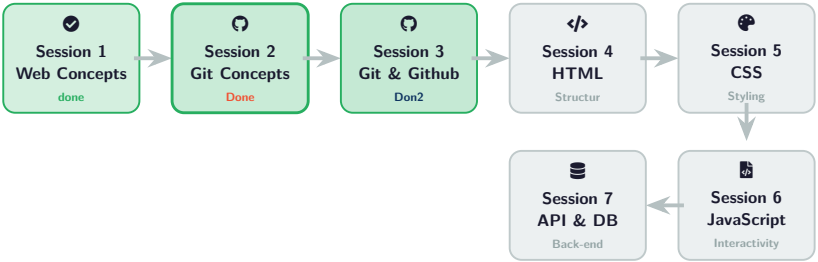
Habits to build from today:

- ✓ **Commit early, commit often**
- ✓ **Write meaningful messages**
- ✓ **Always branch off main**
- ✓ **Pull before you push**
- ✓ **Review before you merge**
- ✗ **Never force-push shared branches**
- ✗ **Never commit directly to main**

Key Concept

Git is not something you learn by reading. It is something you learn by **breaking things and fixing them**. Do it every day — even for personal projects.

What's Coming Next — Full Track Roadmap



Practice Before Session 3

Use Git for **any** project you work on

Open one **Pull Request** on GitHub (even solo)

Try `git log -oneline -graph` on a repo with branches

Bookmark `ohshitgit.com`

Recommended Resources

`git-scm.com/book` — Pro Git (free, full)

`learngitbranching.js.org` — visual

`ohshitgit.com` — emergency fixes

`docs.github.com` — GitHub guides

You Can Use Git Now.

The map made sense. Today you drove.

Next session: HTML — structure, semantics, and your first webpage.

Skills you now have:

- Local workflow: init, add, commit, log
- Remote workflow: push, pull, clone
- Branch, merge, pull request
- Undo: restore, revert, reflog
- Habits that make you a good collaborator

mokeddemsouhil968@gmail.com

datascienceclubusthb

Data Science Club — USTHB

Tech Dept | Dev Web Section

Appendix — Command Quick Reference >_

Command	What it does	Command	What it does
`git init`	Create a new local repository	`git branch`	List local branches
`git clone {url}`	Copy a remote repo locally	`git switch -c {name}`	Create + switch to branch
`git status`	Show working dir & stage state	`git merge {branch}`	Merge branch into current
`git add {file}`	Stage a file	`git push -u origin main`	Push + set upstream
`git add .`	Stage all changes	`git pull`	Fetch + merge remote
`git commit -m {msg}`	Save a snapshot with message	`git fetch`	Download only (no merge)
`git log -oneline`	Compact commit history	`git restore -staged {f}`	Unstage a file
`git diff`	Unstaged changes	`git revert {hash}`	Undo a commit (safe)
`git diff -staged`	Staged vs last commit	`git reflog`	Full action history

Appendix — Common Error Messages Decoded ⚠

Error	What it means & fix
`error: src refspec main does not match any`	You have no commits yet. Create at least one commit first.
`Please tell me who you are`	Run `git config -global user.name` and `user.email`.
`fatal: not a git repository`	You're outside a repo folder. Run `git init` or `cd` into the right folder.
`CONFLICT (content): Merge conflict in {file}`	Two branches edited the same line. Open the file, resolve markers, then `git add` and `git commit`.
`rejected ... non-fast-forward`	Remote has commits you don't have. Run `git pull` first, then push again.

Key Concept

When in doubt: `git status` first. It almost always tells you exactly what to do next.